"Maus programadores se preocupam com o código. Bons programadores se preocupam com as estruturas de dados e seus relacionamentos" (Linus Torvalds).

Módulos

Paulo Lisboa de Almeida

GenAI

UFPR

Conda

Conda

Siga esses passos para instalar o ambiente Conda em sua máquina.

Serve, dentre outras coisas, para evitar conflitos e compartimentar diferentes pacotes Python.

Cuidado!

Faça em casa. Você não vai conseguir instalar nos laboratórios devido às restrições de instalação.

Conda - Derivados Ubuntu

Baixe a última versão do miniconda.

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

Execute o instalador:

```
bash Miniconda3-latest-Linux-x86_64.sh
```

Recarregue o shell, ou execute:

```
source ~/.bashrc
```

Se tudo deu certo, seu shell vai informar o ambiente Conda atual (base):

```
(base) usuario@computador:~$
```

Conda - Ambientes

Para criar um novo ambiente:

```
conda create --name nome_ambiente
```

Para ativar o ambiente:

```
conda activate nome_ambiente
```

Para instalar um pacote no ambiente atual, use preferencialmente:

```
conda install <PACKAGE_NAME>
```

Outros comandos

Para listar todos ambientes disponíveis:

```
conda info --envs
```

Para voltar ao ambiente base:

```
conda activate
```

Módulos

Considere

Considere que todo esse código está em um único script `main.py`.

```python
import numpy as np

def sequencia_fib(qtde_valores):
  resposta = np.empty(qtde_valores, dtype=int)
  a = 0
  b = 1
  aux = b
  idx = 0
  while idx < qtde_valores:
      resposta[idx] = a
      aux = b
      b = a + b
      a = aux
      idx += 1
  return resposta
```

```python
def enesimo_fib(n):
  a = 0
  b = 1
  aux = b
  while n > 0:
      aux = b
      b = a + b
      a = aux
      n -= 1
  return a

def estimar_prop_aurea(n_fibonacci):
  a = 0
  b = 1
  aux = b
  while n_fibonacci > 0:
      aux = b
      b = a + b
      a = aux
      n_fibonacci -= 1
  return b/a

print(sequencia_fib(11))
print(enesimo_fib(10))
print(estimar_prop_aurea(10))
```

Problema

Desejamos organizar as funções

- Manter funções relacionadas em um único arquivo.
- Poder usar as funções em diferentes módulos ou scripts sem a necessidade de dar Control+C / Control+V.
- Organizar o código para facilitar a manutenção.

Módulo

Em Python, um módulo é um arquivo que define um conjunto de funções. Podemos:

- Podemos importar o módulo completo em outro módulo qualquer.

Módulo

Em Python, um módulo é um arquivo que define um conjunto de funções. Podemos:

- Podemos importar o módulo completo em outro módulo qualquer.
- Importar apenas algumas funções do módulo.

Módulo

Em Python, um módulo é um arquivo que define um conjunto de funções. Podemos:

- Podemos importar o módulo completo em outro módulo qualquer.
- Importar apenas algumas funções do módulo.
- O nome do módulo é o mesmo nome do arquivo.
    - Deve ter extensão .py

Estrutura

```
diretorio_projeto
    |
    - main.py
    - fibonacci.py
```

fibonacci.py

```python
import numpy as np

def sequencia_fib(qtde_valores):
  resposta = np.empty(qtde_valores, dtype=int)
  a = 0
  b = 1
  aux = b
  idx = 0
  while idx < qtde_valores:
      resposta[idx] = a
      aux = b
      b = a + b
      a = aux
      idx += 1
  return resposta

def enesimo_fib(n):
  a = 0
  b = 1
  aux = b
  while n > 0:
      aux = b
      b = a + b
      a = aux
      n -= 1
  return a
```

```python
def estimar_prop_aurea(n_fibonacci):
  a = 0
  b = 1
  aux = b
  while n_fibonacci > 0:
      aux = b
      b = a + b
      a = aux
      n_fibonacci -= 1
  return b/a
```

Importando

No arquivo que desejamos usar as funções podemos usar.

```
import fibonacci
```

O módulo será importado, e teremos acesso a todas funções dele.

Acessando

Para acessar uma função do módulo (já importado), use

`nome_modulo.funcao()`

Exemplo:

```
import fibonacci

print(fibonacci.sequencia_fib(11))
```

Criando um alias (pseudônimo)

Durante a importação, podemos criar um alias para o módulo.

Chamaremos o módulo pelo seu alias.

Especialmente útil para simplificar nomes longos.

Sintaxe:

```
import modulo as alias
```

Exemplo

```python
import fibonacci as fib

print(fib.sequencia_fib(11))
print(fib.enesimo_fib(10))
print(fib.estimar_prop_aurea(10))
```

Nome do Módulo

O Python guarda automaticamente o nome do módulo em uma variável chamada `__name__`.

Exemplo:

```
import fibonacci

print(fibonacci.__name__)
```

Importando nomes de função diretamente

É possível importar as funções de um módulo diretamente para o script ou módulo que estamos desenvolvendo.

As funções serão acessíveis como se estivessem sido definidas diretamente no módulo atual.

Sintaxe:

```
from modulo import funcao1, funcao2, …
```

Exemplo

```python
from fibonacci import sequencia_fib, enesimo_fib

enesimo = enesimo_fib(5)
seq = sequencia_fib(5)

print(enesimo)
print(seq)
```

Exemplo

Nesse caso, o módulo fibonacci não é importado para o espaço de nomes local. Apenas as suas funções internas.

```
from fibonacci import sequencia_fib, enesimo_fib

enesimo = enesimo_fib(5)
seq = sequencia_fib(5)
prop = fibonacci.estimar_prop_aurea(5)

print(enesimo)
print(seq)
```

Erro de execução.

Má prática

Não faça isso:

```
from modulo import *
```

O * vai instruir o interpretador a importar todas as funções presentes no módulo.

É um atalho, mas ...

Má prática

Não faça isso:

```
from modulo import *
```

O * vai instruir o interpretador a importar todas as funções presentes no módulo.

É um atalho, mas ...

Má prática

Não faça isso:

```
from modulo import *
```

O * vai instruir o interpretador a importar todas as funções presentes no módulo.

É um atalho, mas ...

- Gera um código difícil de interpretar (quais funções estão acessíveis agora?).
- O que acontece se no nosso arquivo definirmos uma função de nome `func1` e, sem sabermos, dentro de módulo existir uma função de mesmo nome?
- Uma função pode sombrear (shadow) a outra.

Má prática

main.py

```
from mod2 import *
from mod1 import *

imprimeOi()
```

mod1.py

```
def imprimeOi():
    print("Ola mundo do modulo 2")
```

mod2.py

```
def imprimeOi():
    print("Ola mundo do modulo 1")
```

Boa prática

Insira todas as importações de módulos no começo do script.

Exemplo:

```
import fibonacci as fib
import numpy as np
#Demais importações aqui

#Comece os comandos do script aqui
enesimo = fib.enesimo_fib(5)
meu_vetor = np.ones(5, dtype=int)

print(enesimo)
print(meu_vetor)
```

Pacotes

Pacote

Um pacote agrupa múltiplos módulos.

- Organizar o projeto.
- Evitar colisões de nome e variáveis.
  - Exemplo: não precisamos nos preocupar se funções de diferentes pacotes possuem os mesmos nomes.
- Podemos publicar os pacotes para que outras pessoas usem.

Pacote

Para criar um pacote, basta criar um diretório, e inserir os módulos .py nesse diretório.

Exemplo de estrutura:

```
diretorio_projeto
    |
    - main.py
    - matematica_computacao
        |
        - fibonacci.py
        - relacoes.py
```

Pacote

É possível criar quantos pacotes forem necessários no projeto.

É possível ainda aninhar pacotes.

Criar uma estrutura organizacional onde um pacote contém outros pacotes.

Exemplo

```
diretorio_projeto
    |
    - main.py
    - matematica_computacao
        |
        -basica
            |
            - fibonacci.py
            - relacoes.py
        -estatistica
            |
            - funcoes.py
            - testes.py
            …
```

Pacote

Para acessar os pacotes aninhados, basta seguir lógica anterior.

Exemplo com a estrutura anterior.

```
import matematica_computacao.basica.fibonacci as fib
```

Requisitos

Requisitos

No projeto atual, precisamos de pacotes de terceiros.

Por enquanto, do NumPy.

Requisitos

A forma mais simples de especificar os pacotes necessários para executar o projeto é definir um arquivo requirements.txt na raiz do projeto.

Incluir os nomes dos pacotes necessários no arquivo.

Um por linha.

Exemplo

Um exemplo de conteúdo em arquivo requirements.txt:

```
numpy
pandas
```

Instalando

Para instalar as dependências definidas no arquivo requirements.txt:

Garanta que você está no ambiente Conda correto.

Depois, execute: `pip install -r requirements.txt`

Especificando versões

Você pode especificar a versão necessária para a dependência.

Exemplo:

```
numpy==1.26.1
pandas
```

Diz que a versão do NumPy disponível na máquina precisa ser exatamente a 1.26.1.

Especificando versões

Boa prática: use == para indicar uma versão exata apenas quando estritamente necessário.

Isso pode forçar o usuário a ter que instalar uma versão diferente do pacote em sua máquina, mesmo tendo uma versão que poderia cumprir o mesmo propósito.

Exemplo, por que forçar o usuário a instalar o NumPy 1.26.0, se ele já tem a versão 1.26.1 instalada?

Especificando versões

Alguns exemplos de como especificar versões, da menos restritiva para a mais (prefira as menos restritivas quando possível).

numpy	Qualquer versão do NumPy
numpy>=1.26	Qualquer versão igual ou superior à 1.26 do NumPy
numpy>=1.22,<1.27	Qualquer versão igual ou superior à 1.22 e menor que 1.27
numpy==1.26.1	Apenas a versão 1.26.1 do NumPy

Exercícios

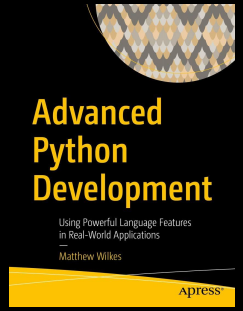
1. Reproduza os comandos dos slides em seu computador, e faça experimentos (ex.: crie seus próprios ambientes Conda. Eles compartilham pacotes?).

Referências

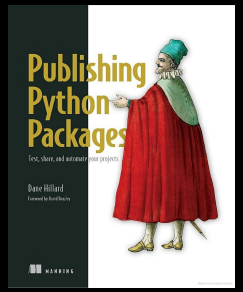
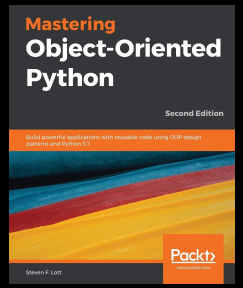
Python Docs: https://docs.python.org

Conda Docs: https://docs.conda.io/en/latest

Wilkes, Matthew. Advanced Python Development: Using Powerful Language Features in Real-World Applications. 2020.



Hillard, Dane. Publishing Python Packages: Test, Share, and Automate Your Projects. 2023.



Lott, Steven F.. Mastering Object-Oriented Python: Build Powerful Applications with Reusable Code Using OOP Design Patterns and Python 3.7. 2 Ed. 2019.